

Efficient first neighbor analysis in large atomic scale models [Article v1.0]

Sébastien Le Roux^{1*†}

¹Institut de Physique et Chimie des Matériaux de Strasbourg, Université de Strasbourg, CNRS UMR 7504, 23 rue de Loess, BP 43, F-67034 Strasbourg Cedex 2, France

This LiveCoMS document is maintained online on GitHub at <https://github.com/Slookeur/Bonds>; to provide feedback, suggestions, or help improve it, please visit the GitHub repository and participate via the issue tracker.

This version dated August 4, 2026

Abstract This tutorial article describes how lattice mathematical properties can be combined with the 3D space pixelation method to evaluate interatomic distances in large atomic scale 3D models with the purpose of searching for first neighbor atoms. It aims to be an educational tool for students and researchers who want to develop structural analysis or 3D visualization tools that requires to implement and compute efficiently the search for first neighbor atoms. Examples codes are provided in C, FORTRAN90 and Python.

***For correspondence:**

sebastien.leroux@ipcms.unistra.fr (SLR)

[†]This author is the sole author of this work

1 Introduction

Every student, every researcher in computational material science, has already spent time calculating interatomic distances. This problem is even likely to be the first one computational material scientists will spend some time over during their studies. As simple as the evaluation of a distance in 3D space could seem to be, the complexity of the problem increases considerably when dealing with the periodicity of non-cubic systems, and even more dramatically with the search for performance that is driving the analysis and the visualization of atomic scale models with more than tens of thousands of atoms.

This manuscript illustrates how lattice mathematical properties can be combined with the pixelation of the model box approach, to offer both a general, symmetry independent, methodology, and, an extremely efficient implementation of the search for first neighbor atoms.

2 Prerequisites

Knowledge and understanding of the mathematics of lattice parameters and atomic coordinates is a prerequisite to the general formulation of the calculation of interatomic bond distances in 3D atomic scale models using **Periodic Boundary Conditions**.

Note that this section can safely be ignored when dealing with non periodic systems.

2.1 Simulation box or lattice parameters

Periodicity can expressed with two different sets of parameters, using either:

1. the box parameters A , B , C and the associated angles α , β and γ
2. the components of the lattice vectors $\vec{a}(a_x, a_y, a_z)$, $\vec{b}(b_x, b_y, b_z)$ and $\vec{c}(c_x, c_y, c_z)$

Lattice vectors can be calculated from the box parameters by:

$$\begin{bmatrix} A & 0.0 & 0.0 \\ \times \cos \gamma & B \times \sin \gamma & 0.0 \\ C \times \cos \beta & C \times L & C \times L^2 \end{bmatrix} \quad (1)$$

With:

$$L = \frac{\cos \alpha - \cos \beta \times \cos \gamma}{\sin \gamma} \quad (2)$$

While box parameters can be calculated from the lattice vectors by:

$$A = |\vec{a}| \quad B = |\vec{b}| \quad C = |\vec{c}| \quad (3)$$

and:

$$\alpha = \frac{\vec{c} \cdot \vec{b}}{B \times C} \quad \beta = \frac{\vec{a} \cdot \vec{c}}{A \times C} \quad \gamma = \frac{\vec{a} \cdot \vec{b}}{A \times B} \quad (4)$$

The lattice volume:

$$V = \vec{a} \cdot (\vec{b} \wedge \vec{c}) = \vec{b} \cdot (\vec{c} \wedge \vec{a}) = \vec{c} \cdot (\vec{a} \wedge \vec{b}) \quad (5)$$

can then be calculated using:

$$V = A \times B \times C \times Z \quad (6)$$

With:

$$Z = \sqrt{1 - \cos^2 \alpha - \cos^2 \beta - \cos^2 \gamma + 2 \cos \alpha \cos \beta \cos \gamma} \quad (7)$$

Knowledge of these properties is a basic requirement, from there it possible to compute transformation matrices that allow to convert Cartesian r to Fractional f coordinates, and vice versa, Fractional f to Cartesian r coordinates. These mathematical tools are extremely useful, if not almost mandatory prerequisites to the calculation, when dealing with non-cubic periodic systems.

2.2 From Cartesian to fractional coordinates

For an atom with Cartesian coordinates (r_x, r_y, r_z) , fractional coordinates (f_x, f_y, f_z) can be calculated using:

$$\begin{pmatrix} f_x \\ f_y \\ f_z \end{pmatrix} = \mathbf{T}_f \times \begin{pmatrix} r_x \\ r_y \\ r_z \end{pmatrix} \quad (8)$$

Where the transformation matrix $\mathbf{T}_f$ is defined as:

$$\mathbf{T}_f = \begin{bmatrix} \frac{1}{A} & -\frac{\cos \gamma}{A \sin \gamma} & \frac{\cos \alpha \cos \gamma - \cos \beta}{A Z \sin \gamma} \\ 0.0 & \frac{1}{B \sin \gamma} & \frac{\cos \alpha \cos \gamma - \cos \beta}{B Z \sin \gamma} \\ 0.0 & 0.0 & \frac{C Z}{\sin \gamma} \end{bmatrix} \quad (9)$$

2.3 From fractional to Cartesian coordinates

Similarly fractional coordinates (f_x, f_y, f_z) can be converted to Cartesian coordinates (r_x, r_y, r_z) using:

$$\begin{pmatrix} r_x \\ r_y \\ r_z \end{pmatrix} = \mathbf{T}_c \times \begin{pmatrix} f_x \\ f_y \\ f_z \end{pmatrix} \quad (10)$$

Where the transformation matrix $\mathbf{T}_c$ is defined as:

$$\mathbf{T}_c = \mathbf{T}_f^{-1} = \begin{bmatrix} A & B \cos \gamma & C \cos \beta \\ 0.0 & B \sin \gamma & C \frac{\cos \alpha - \cos \beta \cos \gamma}{\sin \gamma} \\ 0.0 & 0.0 & \frac{C Z}{\sin \gamma} \end{bmatrix} \quad (11)$$

3 Pixelation of the model box

Frequently used in MD simulations under the name linked cell algorithm(s) [1, 2], the idea of pixelation, or partitioning, of the model box illustrated in this section is mandatory to deal efficiently with searching for first neighbor atoms in large atomic scale models.

Indeed the intuitive way to implement the procedure would be to test every pair $i-j$ of atoms in the model: compute the interatomic distance D_{ij} between i (α) and j (β), and then compared this distance to a cutoff radius $R_{cut}(\alpha, \beta)$, that could appropriately be determined when looking at the radial distribution function $g_{\alpha\beta}(r)$. Then if D_{ij} is smaller or equal to $R_{cut}(\alpha, \beta)$ then atoms i and j are first neighbors, otherwise they are not. For a program which purpose is to render the atomic scale model in 3D space, the result of the analysis would then be to draw, or not, a bond between atoms i and j .

As intuitive and logical as this approach could seem to be, it requires to perform the testing for every pair of atoms in the model. Which, as long as the size of the system remains within the thousand or few thousands of atoms range, could work in a seemingly efficient manner. The time order for the entire analysis is then proportional to $\frac{N \times (N-1)}{2}$, with N the total number of atoms in the model. However as the number of atoms increases, time required to performed the entire analysis increases even more dramatically, soon enough reaching a point where the program will likely seem to be completely frozen.

Therefore another approach is required to optimize the procedure for large atomic scale models, and that is to distinguish atom(s) that are of interest for the purpose of the calculation from atom(s) that are not. This is done by dividing, or partitioning, the model box in smaller parts, or pixels.

Atomic coordinates will allow to associate an atom to a

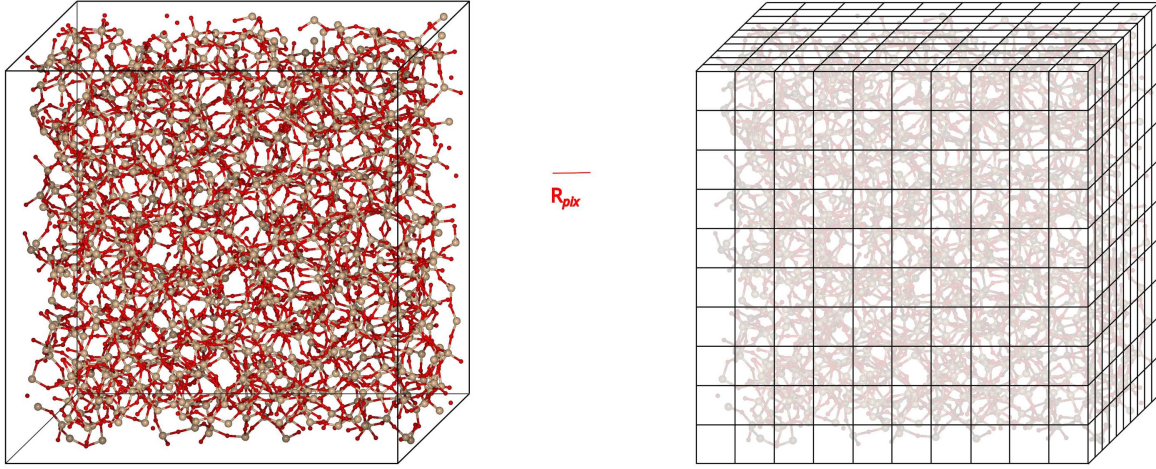


Figure 1. Pixelation of the model box

particular pixel. Then for this particular atom, first neighbors candidates will only be search for in that same pixel and its immediate surrounding pixel neighbors. Ideally the pixel dimension R_{pix} should be close but slightly superior to R_{cut} the cutoff radius used to search for first neighbors atoms, ex: $R_{pix} = R_{cut} + 0.5$.

For any atom in the model this approach will limit the area of interest to the smallest possible size. Using this methodology the time order for the entire analysis becomes proportional to $N_p \times \frac{N_c \times (N_c - 1)}{2}$, with N_p is the total number of pixels in the grid, and N_c is the average number of atom(s) in the pixel and its 26 surrounding neighbor pixels: $N_c \ll N_p \ll N$.

The idea behind this approach is illustrated in figure 1.

3.1 Without Periodic Boundary Conditions

The number of pixels on each axis, $n_p(x)$, $n_p(y)$ and $n_p(z)$, are calculated using:

$$n_p(axis) = \left\lceil \frac{D_{max}(axis)}{R_{pix}} \right\rceil \quad (12)$$

Where $D_{max}(axis)$ is the maximum interatomic distance separating two atoms on *axis*, and $R_{pix} \simeq R_{cut}$ is the pixel dimension, close to the cutoff distance that separates first neighbor atoms.

Providing a model box, with parameters A, B, and C, encompassing the entire model could prove useful here, allowing the simplifications:

$$D_{max}(x) = A, \quad D_{max}(y) = B \quad \text{and} \quad D_{max}(z) = C \quad (13)$$

Otherwise calculations to determine D_{max} for each axis are needed. It is then required to test each and every pair of

atomic coordinates in the model on *x*, *y* and *z*. However as long as only subtractions and min/max comparisons are involved calculation time will remain completely acceptable.

Then the total number of pixels in the model box, *pixels*, is calculated using:

$$pixels = n_p(x) \times n_p(y) \times n_p(z) \quad (14)$$

For an atom *at* with Cartesian coordinates (r_x , r_y , r_z) in the model, corresponding pixel indices in the pixel grid (p_x , p_y , p_z) can be calculated using:

$$p_{axis} = \left\lceil \frac{r_{axis} - \min_{axis}}{R_{pix}} \right\rceil \quad (15)$$

With:

$$p_{axis} \in [0, n_p(axis) - 1] \quad (16)$$

Where $\min_{axis}$ is the lowest value for any atomic coordinates in the model on *axis*.

The pixel number for *at*, between 0 and *pixels* – 1, in entire the pixel grid, P_{id} is calculated using:

$$P_{id}(at) = p_x + n_p(x) \times p_y + [n_p(x) \times n_p(y)] \times p_z \quad (17)$$

With:

$$P_{id} \in [0, pixels - 1] \quad (18)$$

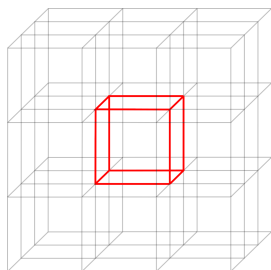
First neighbors list is calculated as follow:

1. The pixel position (p_x , p_y , p_z) for every atom (r_x , r_y , r_z) in the model is to be calculated so that each atom can be assigned a pixel number in the grid.
2. Accordingly a list of atom containing pixels is created, the list of atom(s) in each pixel being stored.

3. First neighbor(s) for an atom will then be search for in the pixel this atom belong to, and, in its surrounding pixel neighbors only, all other pixel(s) being safely ignored.

The list of pixel neighbors is constructed using the mathematical relationships between each pixel index in the grid, two cases must considered:

- Pixel inside the pixel grid:

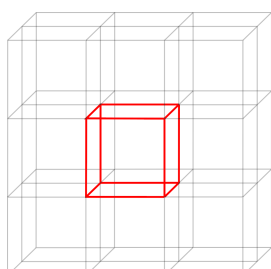


Self + 26 neighbors

- Pixel on the boundary of the pixel grid :

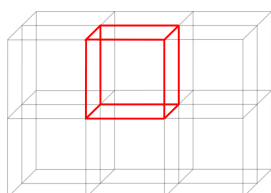
$$p_{axis} = 0 \text{ or } p_{axis} = n_p(axis) - 1$$

Face of the pixel grid



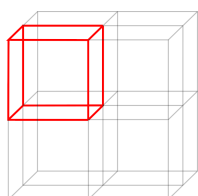
Self + 17 neighbors

Edge of the pixel grid



Self + 11 neighbors

Corner of the pixel grid



Self + 7 neighbors

Pixel neighbors are determined as illustrated in figure 2:

Note that this kind of approach only makes sense if the number of pixels in the grid is high enough so that all pixels are not neighbors.

3.2 With Periodic Boundary Conditions

Non-cubic symmetries make it more complicated to offer a general methodology to deal with periodic systems:

1. Evaluate the number of pixel(s) on each axis, $n_p(x)$, $n_p(y)$ and $n_p(z)$, using equations 12 and 13.
2. Convert atomic Cartesian coordinates to fractional coordinates using $\mathbf{T}_f$.

Using fractional coordinates is the easiest way to propose a general methodology to compute interatomic distances in the model. Indeed this requires to consider the periodicity of the system, and, in the case of non-cubic symmetry transformations, could be tricky when using Cartesian coordinates. Working with fractional coordinates is much easier since in that case corrections are performed simply adding or subtracting multiples of 1.0 on any fractional direction.

- Convert Cartesian coordinates to fractional coordinates (f_x, f_y, f_z) using Eq. 8.
- Compute corrected fractional coordinates $(f_{c,x}, f_{c,y}, f_{c,z})$ inside the model box:

$$f_{c,axis} = f_{axis} - \lfloor f_{axis} \rfloor \quad (19)$$

With:

$$0 \leq f_{c,axis} < 1 \quad (20)$$

3. Pixel positions (p_x, p_y, p_z) are determined using the atom's corrected fractional coordinates $(f_{c,x}, f_{c,y}, f_{c,z})$:

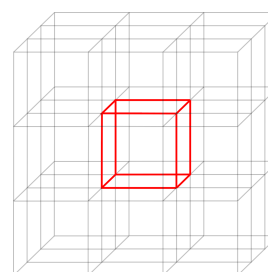
$$p_{axis} = \lfloor f_{c,axis} \times n_p(axis) \rfloor \quad (21)$$

With:

$$p_{axis} \in [0, n_p(axis) - 1] \quad (22)$$

4. Determine each pixel neighbors:

- Pixel inside the pixel grid:



Self + 26 neighbors, no PBC transformation required.

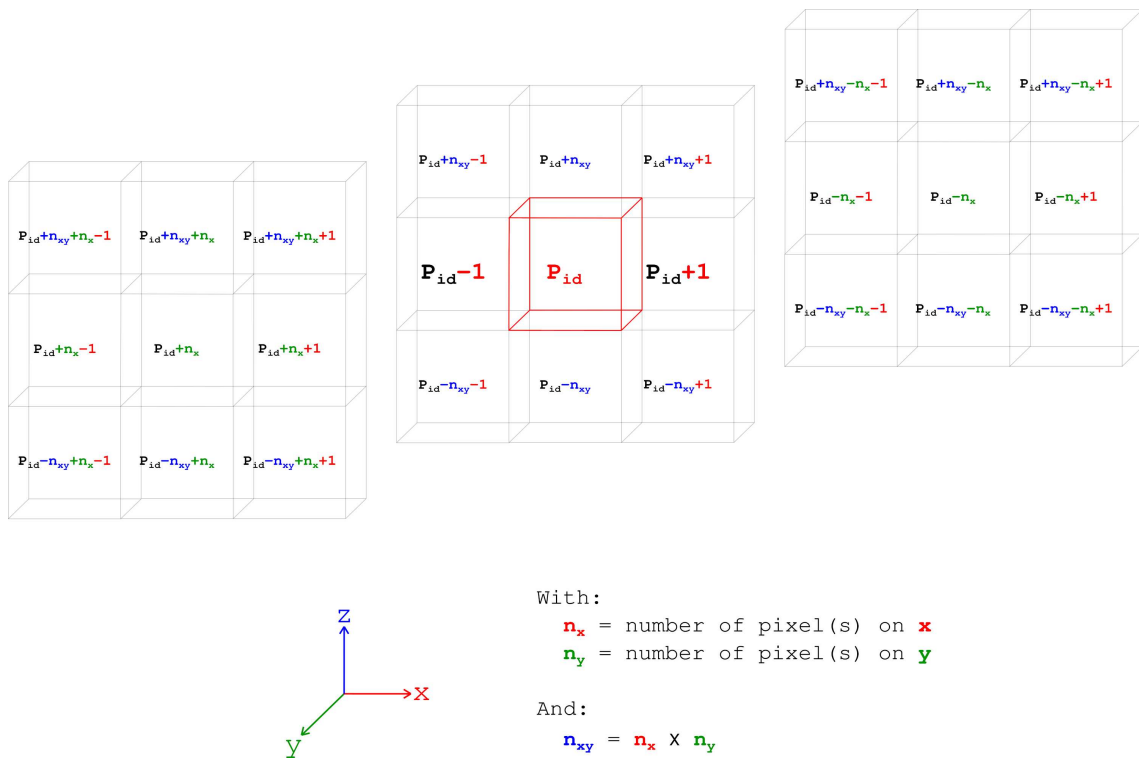
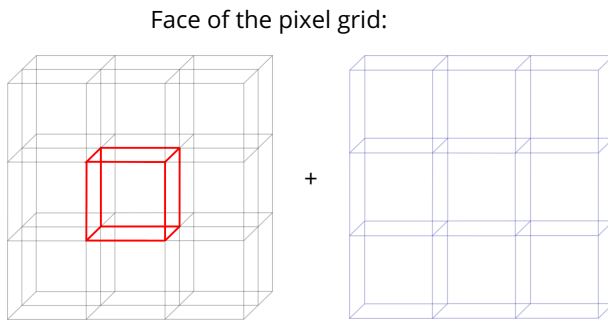


Figure 2. Finding pixel neighbors for pixel P_{id} : operation(s) on each axis are illustrated in the appropriate color(s)

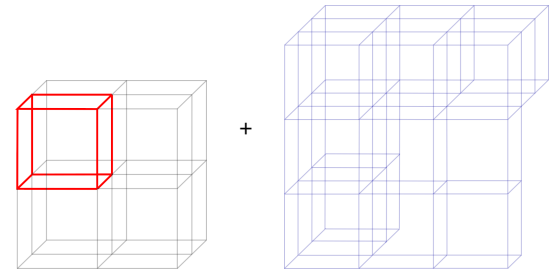
- Pixel on the boundary of the pixel grid :
 $p_{axis} = 0$ or $p_{axis} = n_p(axis) - 1$

Self + 11 + 15 neighbors using PBC

Corner of the pixel grid:

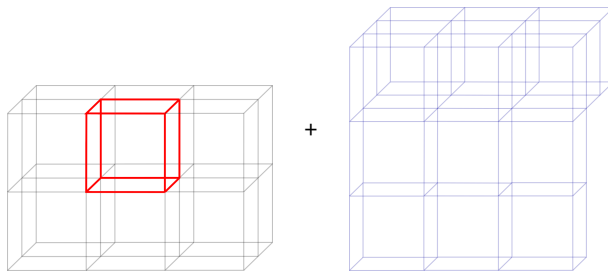


Self + 17 + 9 neighbors using PBC



Self + 7 + 19 neighbors using PBC

Edge of the pixel grid:



For a pixel with number p_{id} with pixel coordinates (p_x, p_y, p_z) inside the pixel grid, pixel neighbors are determined as illustrated in figure 2. For a pixel on the boundary of the grid, as the 3 cases described above, that means if $p_{axis} = 0$ or $p_{axis} = n_p(axis) - 1$, then it is possible to determine its PBC pixel neighbors by applying a shift to the pixel neighbors number calculated as illustrated in fig. 2.

In the in the 3 directions of space:

if $p_x = 0$ shift all $p_x - 1$ pixels by $n_p(x)$

if $p_x = n_p(x) - 1$ shift all $p_x + 1$ pixels by $-n_p(x)$

if $p_y = 0$ shift all $p_y - 1$ pixels by $n_p(xy)$

if $p_y = n_p(y) - 1$ shift all $p_y + 1$ pixels by $-n_p(xy)$

if $p_z = 0$ shift all $p_z - 1$ pixels by $n_p(xyz)$

if $p_z = n_p(z) - 1$ shift all $p_z + 1$ pixels by $-n_p(xyz)$

With:

$$n_p(xy) = n_p(x) \times n_p(y) \quad (23)$$

And:

$$n_p(xyz) = n_p(xy) \times n_p(z) \quad (24)$$

- When searching for first neighbor atoms between atoms i and j of the same pixel or one of its surrounding pixel neighbor, compute the interatomic distance D_{ij} between i and j using their corrected fractional coordinates:

$$f_{ij}(axis) = f_{c,axis}(i) - f_{c,axis}(j) \quad (25)$$

$$f_{c,ij}(axis) = f_{ij}(axis) - \lfloor f_{ij}(axis) \rfloor \quad (26)$$

$$\vec{r}_{ij} = \mathbf{T}_c \times \vec{f}_{c,ij} \quad (27)$$

$$\text{with } \vec{r}_{ij} = \begin{pmatrix} r_{ij}(x) \\ r_{ij}(y) \\ r_{ij}(z) \end{pmatrix} \quad (28)$$

$$\text{and } \vec{f}_{c,ij} = \begin{pmatrix} f_{c,ij}(x) \\ f_{c,ij}(y) \\ f_{c,ij}(z) \end{pmatrix} \quad (29)$$

$$D_{ij} = |\vec{r}_{ij}| \quad (30)$$

Note that some further calculation time can be spared if D_{ij}^2 only is calculated so that no squared root calculation is performed to obtain D_{ij} . Therefore in that case D_{ij}^2 must be compared to R_{cut}^2 .

As mentioned in the previous section this approach only makes sense when the number of pixels in the grid is high enough so that all pixels are not neighbors. In the case where PBC are applied this means that the number of pixels on one dimension, x , y , or z should be higher than 3.

4 Limitations

Overall the pixelation, or partitioning, of the model box approach is extremely efficient. However some issues might

arise when dealing with a very large system containing very few atoms, which could happen if:

- No periodicity is involved but the model is very large and describes a single, or few, isolated molecule(s) that are far apart.
- The description of the model relies on a very large box, usually with a simple cubic periodicity, that contains a single or few isolated molecule.

In such rare cases the number density, in atoms / $\text{\AA}^3$, can be very low (< 0.01 atom / $\text{\AA}^3$), but the number of pixels required to build the entire pixel grid can be too high for the memory allocation(s) required to perform the analysis. To overcome this issue the size of the pixel R_{pix} can be increased, to a value significantly higher than the cutoff radius R_{cut} , so that the density of atom(s) per pixel can be adjusted to match a proper target value to ensure that memory allocation will not be an issue, and that the analysis will be successful. By experience a value between 1.5 and 2 atoms per pixel is a safe window to avoid any issue in these peculiar cases.

5 Example codes

Commented codes that illustrate the entire work procedure are provided in the manuscript appendixes:

- C codes: A.1 through A.8.
- FORTRAN90 codes: B.1 through B.8.
- Python codes: C.1 through C.8.

Note that this Python code is provided to illustrate the entire implementation in Python, but for that particular programming language several Python libraries already exist and can be used as fronted to simplify the complete coding (ex: [ASE](#), [Pysic](#), [MDAnalysis](#)), however in that case the pixelation approach is rarely coded in pure Python language ([Pysic](#) using FORTRAN, [MDAnalysis](#) using C).

6 Further optimizations

The analysis time can be reduced using MPI and/or OpenMP parallel programming. Several scenario, or approaches, can be envisioned depending on the size of the system in number of atoms and/or the number of configuration (MD steps):

- Single (MPI or OpenMP): atomic coordinates or pixels can be distributed over the CPU and/or CPU cores.
- Multiples (MPI or OpenMP): configurations can be distributed over the CPU and/or the CPU cores.
- Multiples (MPI and OpenMP): with hybrid parallelization configurations can be distributed over the CPU, and atomic coordinates or pixels can be distributed over the CPU cores.

Ideally the code would provide the option to switch between one and the other approach based on the number of configurations and or atoms in the system.

Note that this is the case of the **atomes** software [3] that implements an adaptive OpenMP programming distributing either the atomic coordinates or the MD steps on the CPU cores.

7 Conclusion

The general methodology behind efficient first neighbor(s) analysis in any kind of atomic scale models was described.

It was illustrated that the understanding of lattice mathematics can simplify the evaluation of interatomic bond distances independently of the periodicity of the system, and that the particular idea of the pixelation, or partitioning, of the model box, is a prerequisite to any modern implementation of this analysis.

The methodology described in this article is the one implemented in the **atomes** software [3].

Checklist

For a more detailed description of author contributions, see the GitHub issue tracking and changelog at <https://github.com/Slookeur/Bonds>.

Potentially Conflicting Interests

No conflicting interests.

Author Information

ORCID:

Sébastien Le Roux: [0000-0002-1912-6960](https://orcid.org/0000-0002-1912-6960)

References

- [1] Allen MP, Tildesley DJ. Computer Simulation of Liquids. Oxford University Press; 1989.
- [2] Welling U, Germano G. Computer Physics Communications. 2011; 182(3):611–615. <https://doi.org/10.1016/j.cpc.2010.11.002>.
- [3] Le Roux S. Comp. Mat. Sci.. 2025; 253:113805. <https://doi.org/10.1016/j.commatsci.2025.113805>.

SEARCHING FOR FIRST NEIGHBOR ATOMS

Non-periodic systems

- ☐ Choose a cutoff radius R_{cut} to determine first neighbor atoms, and the associated pixel dimension R_{pix}
- ☐ Find the maximum distances, $D_{max}(x)$, $D_{max}(y)$ and $D_{max}(z)$ that separate 2 atoms on x , y and z
- ☐ Determine the number of pixels, $n_p(axis)$, on x , y and z using $n_p(axis) = \left\lceil \frac{D_{max}(axis)}{R_{pix}} \right\rceil$
- ☐ Create a pixel grid for a virtual box with dimensions $n_p(x)$, $n_p(y)$ and $n_p(z)$ on x , y and z axis respectively.
- ☐ Using its Cartesian coordinates, $r(at)$, associate each atom at with a pixel position in the grid
 - ☐ Pixel position on each $axis$: $p_{axis} = \left\lfloor \frac{r_{at}(axis) - min_{axis}}{R_{pix}} \right\rfloor$
 - ☐ Pixel ID number in the grid: $P_{id}(at) = p_x + n_p(x) \times p_y + [n_p(x) \times n_p(y)] \times p_z$
- ☐ Associate each pixel with its surrounding neighbors in the grid
- ☐ For all pixels in the grid
 - ☐ Search for neighbors between the atoms of pixel pix as well as the atoms of its surrounding pixel neighbors pjx
 - ☐ For 2 atoms i in pixel pix and j in pixel pjx
 - ☐ Evaluate r_{ij} on x , y and z using atoms i and j Cartesian coordinates
 - ☐ Compute and return D_{ij}^2
 - ☐ Compare D_{ij}^2 to R_{cut}^2 to determine if i and j are first neighbors

Periodic systems

- ☐ Choose a cutoff radius R_{cut} to determine first neighbor atoms, and the associated pixel dimension R_{pix}
- ☐ Use the lattice parameters as maximum interatomic distances on x , y and z : $D_{max}(x) = A$, $D_{max}(y) = B$ and $D_{max}(z) = C$
- ☐ Determine the number of pixels, $n_p(axis)$, on x , y and z using $n_p(axis) = \left\lceil \frac{D_{max}(axis)}{R_{cut}} \right\rceil$
- ☐ Create a pixel grid with dimensions $n_p(x)$, $n_p(y)$ and $n_p(z)$ on x , y and z axis respectively.
- ☐ Convert all atom Cartesian coordinates to fractional coordinates f using $\mathbf{T}_f$
- ☐ Compute the corrected fractional coordinates f_c to ensure that all atoms are in the unit cell $f_c = f - \lfloor f \rfloor$
- ☐ Using its correct fractional coordinates, $f_c(at)$, associate each atom at with a pixel position in the grid
 - ☐ Pixel position on each $axis$: $p_{axis} = \lfloor f_{c,axis}(at) \times n_p(axis) \rfloor$
 - ☐ Pixel ID number in the grid: $P_{id}(at) = p_x + n_p(x) \times p_y + [n_p(x) \times n_p(y)] \times p_z$
- ☐ Associate each pixel with its surrounding neighbors in the grid, if needed apply PBC transformation(s)
- ☐ For all pixels in the grid
 - ☐ Search for neighbors between the atoms of pixel pix as well as the atoms of its surrounding pixel neighbors pjx
 - ☐ For 2 atoms i in pixel pix and j in pixel pjx
 - ☐ Evaluate f_{ij} on x , y and z using the atom i and j corrected fractional coordinates f_c
 - ☐ Correct f_{ij} to $f_{c,ij}$ for potential PBC correction(s): $f_{c,ij} = f_{ij} - \lfloor f_{ij} \rfloor$
 - ☐ Transform back $f_{c,ij}$ to Cartesian coordinates r_{ij} using $\mathbf{T}_c$
 - ☐ Compute and return D_{ij}^2
 - ☐ Compare D_{ij}^2 to R_{cut}^2 to determine if i and j are first neighbors

Appendix A Commented C code

A.1 C code: data structures and global variables

```

1 // global data structures and variables used in the next code sections
2 #include <math.h>
3
4 #define TRUE 1
5 #define FALSE 0
6
7 // atom in pixel data structure
8 typedef struct pixel_atom pixel_atom;
9 struct pixel_atom
10 {
11     int atom_id;           // the atom ID
12     float coord[3];       // the atom coordinates on x, y and z
13 };
14
15 // pixel data structure
16 typedef struct pixel pixel;
17 struct pixel
18 {
19     int pid;               // the pixel number
20     int p_xyz[3];          // the pixel coordinates in the grid
21     bool tested;           // was the pixel checked already
22     int patoms;            // number of atom(s) in pixel
23     pixel_atom * pix_atoms; // list of atom(s) in the pixel, to be allocated
24     int neighbors;         // number of neighbors for pixel
25     int pixel_neighbors[27]; // the list of neighbor pixels, maximum 27
26 };
27
28 // pixel grid data structure
29 typedef struct pixel_grid pixel_grid;
30 struct pixel_grid
31 {
32     int pixels;            // total number of pixels in the grid
33     int n_pix[3];          // number of pixel(s) on each axis
34     int n_xy;              // number of pixels in the plan xy
35     pixel * pixel_list;    // pointer to the pixels, to be allocated
36 };
37
38 // bond distance data structure
39 typedef struct distance distance;
40 struct distance
41 {
42     float length;          // the distance in \AA\ squared
43     float Rij[3];          // vector components of x, y and z
44 };
45
46 /*
47  the following are considered to be provided by the user
48 */
49
50 // model description
51 int atoms;                // the total number of atom(s)
52 float ** c_coord;         // list of Cartesian coordinates: c_coord[atoms][3]
53 float cutoff;             // the cutoff to define atomic bond(s)
54 float cutoff_squared;     // squared value for the cutoff
55
56 // model box description, if PBC are applied
57 float volume;             // the lattice volume
58 float l_params[3];        // lattice a, b and c
59 float cart_to_frac[3][3]; // Cartesian to fractional coordinates matrix
60 float frac_to_cart[3][3]; // fractional to Cartesian coordinates matrix

```

A.2 C code: set periodic boundary condition pixel shift

```

1 void set_pbc_shift (pixel_grid * grid, int pixel_coord[3], int pbc_shift[3][3][3])
2 {
3     int x_pos, y_pos, z_pos; // loop iterators
4     for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
5     {
6         for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
7         {
8             for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
9             {
10                 pbc_shift[x_pos][y_pos][z_pos] = 0;           // initialization without any shift
11             }
12         }
13     }
14     if ( pixel_coord[0] == 0 )                                // pixel position on 'x' is min
15     {
16         for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
17         {
18             for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
19             {
20                 pbc_shift[0][y_pos][z_pos] = grid->n_pix[0];
21             }
22         }
23     }
24     else if ( pixel_coord[0] == grid->n_pix[0] - 1 )          // pixel position on 'x' is max
25     {
26         for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
27         {
28             for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
29             {
30                 pbc_shift[2][y_pos][z_pos] = - grid->n_pix[0];
31             }
32         }
33     }
34     if ( pixel_coord[1] == 0 )                                // pixel position on 'y' is min
35     {
36         for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
37         {
38             for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
39             {
40                 pbc_shift[x_pos][0][z_pos] += grid->n_xy;
41             }
42         }
43     }
44     else if ( pixel_coord[1] == grid->n_pix[1] - 1 )          // pixel position on 'y' is max
45     {
46         for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
47         {
48             for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
49             {
50                 pbc_shift[x_pos][2][z_pos] -= grid->n_xy;
51             }
52         }
53     }
54     if ( pixel_coord[2] == 0 )                                // pixel position on 'z' is min
55     {
56         for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
57         {
58             for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
59             {
60                 pbc_shift[x_pos][y_pos][0] += grid->pixels;
61             }
62         }
63     }
64     else if ( pixel_coord[2] == grid->n_pix[2] - 1 )          // pixel position on 'z' is max
65     {
66         for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
67         {
68             for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
69             {
70                 pbc_shift[x_pos][y_pos][2] -= grid->pixels;
71             }
72         }
73     }
74 }

```

A.3 C code: add atom to pixel

```

1 void add_atom_to_pixel (pixel * the_pixel, int pixel_coord[3], int atom_id, float atom_coord[3])
2 {
3     int axis;
4     if (! the_pixel->patoms)
5     {
6         // if the pixel do not contains any atom yet, then save its coordinates in the grid
7         for ( axis = 0 ; axis < 3 ; axis ++ )
8         {
9             the_pixel->p_xyz[axis] = pixel_coord[axis];
10        }
11        // allocate the memory to store the first pixel_atom information
12        the_pixel->pix_atoms = malloc(sizeof*the_pixel->pix_atoms);
13    }
14    else
15    {
16        // otherwise reallocate memory to store the new pixel_atom information
17        the_pixel->pix_atoms = realloc(the_pixel->pix_atoms, (the_pixel->patoms+1)*sizeof*the_pixel->pix_atoms);
18    }
19    the_pixel->pix_atoms[the_pixel->patoms].atom_id = atom_id;
20    for ( axis = 0 ; axis < 3 ; axis ++ )
21    {
22        the_pixel->pix_atoms[the_pixel->patoms].coord[axis] = atom_coord[axis];
23    }
24    // increment the number of atom(s) in the pixel
25    the_pixel->patoms ++;
26 }

```

A.4 C code: adjusting the number of pixels in the grid

```

1 float adjust_pixel_numbers (bool use_pbc, pixel_grid * grid, float cmin[3], float cmax[3], float pixel_size)
2 {
3
4     int axis;                // loop iterator axis id (1=x, 2=y, 3= z)
5     float targetdp = 1.85;    // target atom density per pixel
6     float rhonum;             // number density
7     float rhopix;             // number density by pixel
8     float mpsize;             // modifier of the pixel size
9
10    rhonum = atoms;
11    if ( use_pbc )
12    {
13        // if PBC are used then use the exact volume
14        rhonum = rhonum / volume;
15    }
16    else
17    {
18        // otherwise approximate a cubic box
19        for ( axis = 0 ; axis < 3 ; axis ++ )    // For x, y and z
20        {
21            rhonum /= (cmax[axis] - cmin[axis]);
22        }
23    }
24
25    // if the number density is < 0.01 atom / Å³
26    if (rhonum < 0.01)
27    {
28        rhopix = atoms;
29        for ( axis = 0 ; axis < 3 ; axis ++ )    // For x, y and z
30        {
31            rhopix /= grid->n_pix[axis];
32        }
33        mpsize = pow (targetdp/rhopix, 1.0/3.0);
34        if (mpsize > 1.0)
35        {
36            // we modify only the pixel size if the cutoff increases
37            // otherwise we would increase the number of pixels
38            pixel_size = pixel_size*mpsize;
39            for ( axis = 0 ; axis < 3 ; axis ++ )
40            {
41                if ( use_pbc )
42                {
43                    grid->n_pix[axis] = (int)(l_params[axis] / pixel_size);
44                }
45                else
46                {
47                    grid->n_pix[axis] = (int)((cmax[axis] - cmin[axis]) / pixel_size);
48                }
49            }
50        }
51    }
52    for ( axis = 0 ; axis < 3 ; axis ++ )    // For x, y and z
53    {
54        // correction if the number of pixel(s) on 'axis' is too small
55        grid->n_pix[axis] = (grid->n_pix[axis] < 3) ? 1 : grid->n_pix[axis];
56    }
57    return pixel_size;
58 }

```

A.5 C code: preparation of the pixel grid

```

1 pixel_grid * prepare_pixel_grid (bool use_pbc)
2 {
3     pixel_grid * grid;           // pointer to the pixel grid to create
4     int axis;                    // loop iterator axis id (0=x, 1=y, 2=z)
5     int aid;                     // loop iterator atom number (0, atoms--1)
6     int pixel_num;               // pixel number in the grid
7     int pixel_pos[3];            // pixel coordinates in the grid
8     float cmin[3], cmax[3];      // float coordinates min, max values
9     float f_coord[3];           // float fractional coordinates
10    float pixel_size;            // the size of the pixel
11
12    grid = malloc(sizeof*grid);
13    pixel_size = cutoff + 0.5;    // set a pixel size slightly higher than the cutoff
14    if ( use_pbc )                // using periodic boundary conditions
15    {
16        for ( axis = 0 ; axis < 3 ; axis ++ )    // For x, y and z
17        {
18            grid->n_pix[axis] = (int)(l_params[axis] / pixel_size); // number of pixel(s) on 'axis'
19        }
20    }
21    else                          // without periodic boundary conditions
22    {
23        for ( axis = 0 ; axis < 3 ; axis ++ ) cmin[axis] = cmax[axis] = c_coord[0][axis];
24        for ( aid = 1 ; aid < atoms ; aid ++ )    // For all atoms
25        {
26            for ( axis = 0 ; axis < 3 ; axis ++ )    // For x, y and z
27            {
28                cmin[axis] = min(cmin[axis], c_coord[aid][axis]);
29                cmax[axis] = max(cmax[axis], c_coord[aid][axis]);
30            }
31        }
32        for ( axis = 0 ; axis < 3 ; axis ++ )    // For x, y and z
33        {
34            grid->n_pix[axis] = (int)((cmax[axis] - cmin[axis])/pixel_size); // number of pixel(s) on 'axis'
35        }
36    }
37    pixel_size = adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size);
38
39    grid->n_xy = grid->n_pix[0] * grid->n_pix[1]; // number of pixels on the plan 'xy'
40    grid->pixels = grid->n_xy * grid->n_pix[2]; // total number of pixels in the grid
41    grid->pixel_list = malloc (grid->pixels*sizeof*grid->pixel_list);
42    for ( pixel_num = 0 ; pixel_num < grid->pixels ; pixel_num ++ )
43    {
44        grid->pixel_list[pixel_num].pid = pixel_num;
45        grid->pixel_list[pixel_num].patoms = 0;
46        grid->pixel_list[pixel_num].tested = FALSE;
47    }
48    if ( use_pbc )                // using periodic boundary conditions
49    {
50        for ( aid = 0 ; aid < atoms ; aid ++ )    // for all atoms
51        {
52            // with 'matrix_multiplication' a user defined function to perform the operation
53            matrix_multiplication (f_coord, cart_to_frac, c_coord[aid]);
54            for ( axis = 0 ; axis < 3 ; axis ++ )    // for x, y and z
55            {
56                f_coord[axis] = f_coord[axis] - floorf(f_coord[axis]);
57                pixel_pos[axis] = (int)(f_coord[axis] * grid->n_pix[axis]);
58            }
59            pixel_num = pixel_pos[0] + pixel_pos[1] * grid->n_pix[0] + pixel_pos[2] * grid->n_xy;
60            add_atom_to_pixel (& grid->pixel_list[pixel_num], pixel_pos, aid, f_coord);
61        }
62    }
63    else                          // without periodic boundary conditions
64    {
65        for ( aid = 0 ; aid < atoms ; aid ++ )    // for all atoms
66        {
67            for ( axis = 0 ; axis < 3 ; axis ++ )    // for x, y and z
68            {
69                pixel_pos[axis] = (grid->n_pix[axis] == 1) ? 0 : (int)((c_coord[aid][axis] - cmin[axis])/pixel_size);
70            }
71            pixel_num = pixel_pos[0] + pixel_pos[1] * grid->n_pix[0] + pixel_pos[2] * grid->n_xy;
72            add_atom_to_pixel (& grid->pixel_list[pixel_num], pixel_pos, pixel_pos, aid, c_coord[aid]);
73        }
74    }
75    return grid;
76 }

```

A.6 C code: finding pixel neighbors

```

1 // finding neighbor pixels for pixel in the grid
2 // - bool use_pbc : flag to set if PBC are used or not
3 // - pixel_grid * the_grid : pointer to the pixel grid
4 // - pixel * the_pix : pointer to the pixel with neighbors to be found
5 void find_pixel_neighbors (bool use_pbc, pixel_grid * the_grid, pixel * the_pix)
6 {
7     int axis; // loop iterator axis id (1=x, 2=y, 3= z)
8     int x_pos, y_pos, z_pos; // neighbor position on x, y and z
9     int l_start[3] = { 0, 0, 0 }; // loop iterators starting value
10    int l_end[3] = { 3, 3, 3 }; // loop iterators ending value
11    int pmod[3] = { -1, 0, 1 }; // position modifiers
12    int nnp; // number of neighbors for pixel
13    int nid; // neighbor id for pixel
14    int pbc_shift[3][3][3]; // shift for pixel neighbor number due to PBC
15    bool boundary = FALSE; // is pixel on the boundary of the grid
16    bool keep_neighbor = TRUE; // keep or not neighbor during analysis
17
18    if ( use_pbc )
19    {
20        // set PBC correction based on grid and pixel data
21        set_pbc_shift (the_grid, the_pix->p_xyz, pbc_shift);
22    }
23    else
24    {
25        for ( axis = 0 ; axis < 3 ; axis ++ )
26        {
27            // if min or max on 'axis' then 'the_pix' is on the edge of the box
28            if ( the_pix->p_xyz[axis] == 0 || the_pix->p_xyz[axis] == the_grid->n_pix[axis]-1 ) boundary = TRUE;
29        }
30    }
31    for ( axis = 0 ; axis < 3 ; axis ++ )
32    {
33        if ( the_grid->n_pix[axis] == 1 )
34        {
35            // in the grid there is a single pixel in the 'axis' direction
36            l_start[axis] = 1;
37            l_end[axis] = 2;
38        }
39    }
40    nnp = 0;
41    for ( x_pos = l_start[0] ; x_pos < l_end[0] ; x_pos ++ )
42    {
43        for ( y_pos = l_start[1] ; y_pos < l_end[1] ; y_pos ++ )
44        {
45            for ( z_pos = l_start[2] ; z_pos < l_end[2] ; z_pos ++ )
46            {
47                keep_neighbor = TRUE;
48                if ( ! use_pbc && boundary )
49                {
50                    if ( ( the_pix->p_xyz[0] == 0 && x_pos == 0 ) || ( the_pix->p_xyz[0] == the_grid->n_pix[0]-1 && x_pos == 2 ) )
51                    {
52                        keep_neighbor = FALSE;
53                    }
54                    else if ( ( the_pix->p_xyz[1] == 0 && y_pos == 0 ) || ( the_pix->p_xyz[1] == the_grid->n_pix[1]-1 && y_pos == 2 ) )
55                    {
56                        keep_neighbor = FALSE;
57                    }
58                    else if ( ( the_pix->p_xyz[2] == 0 && z_pos == 0 ) || ( the_pix->p_xyz[2] == the_grid->n_pix[2]-1 && z_pos == 2 ) )
59                    {
60                        keep_neighbor = FALSE;
61                    }
62                }
63                if ( keep_neighbor )
64                {
65                    // evaluating neighbor pixel number in the grid
66                    nid = the_pix->pid + pmod[x_pos] + pmod[y_pos] * the_grid->n_pix[0] + pmod[z_pos] * the_grid->n_xy;
67                    // corrections if required in the case where PBC are applied
68                    if ( use_pbc ) nid += pbc_shift[x_pos][y_pos][z_pos];
69                    the_pix->pixel_neighbors[nnp] = nid;
70                    nnp ++ ;
71                }
72            }
73        }
74    }
75    // total number of neighbors for pixel 'the_pix'
76    the_pix->neighbors = nnp;
77 }

```

A.7 C code: inter-atomic distance calculation

```

1 // evaluating the interatomic distance between 2 pixel atoms
2 // - bool use_pbc      : flag to set if PBC are used or not
3 // - pixel_atom * at_i : pointer to first pixel atom
4 // - pixel_atom * at_j : pointer to second pixel atom
5 distance evaluate_distance (bool use_pbc, pixel_atom * at_i, pixel_atom * at_j)
6 {
7     int axis;           // axis loop iterator
8     float u, v;         // float parameters
9     distance dist;      // distance data to store calculation results
10    for ( axis = 0 ; axis < 3 ; axis ++ )
11    {
12        dist.Rij[axis] = at_i->coord[axis] - at_j->coord[axis];
13    }
14    if ( use_pbc )
15    {
16        // then the pixel_atom's coordinates are in corrected fractional format
17        for ( axis = 0 ; axis < 3 ; axis ++ )
18        {
19            dist.Rij[axis] = dist.Rij[axis] - roundf(dist.Rij[axis]);
20        }
21        // transform back to Cartesian coordinates
22        // with 'matrix_multiplication' a user defined function to perform the operation
23        matrix_multiplication (dist.Rij, frac_to_cart, dist.Rij);
24    }
25    dist.length = 0.0;
26    for ( axis = 0 ; axis < 3 ; axis ++ )
27    {
28        dist.length += dist.Rij[axis] * dist.Rij[axis];
29    }
30    // returning the 'distance' data structure that contains:
31    // - the squared value for Dij: no time consuming square root calculation !
32    // - the components of the distance vector on x, y and z
33    return dist;
34 }

```


A.8 C code: pixel search for first neighbor atoms

```

1 // searching for first neighbor atoms using the grid pixelation/partitioning method
2 // - bool use_pbc : flag to set if PBC are used or not
3 void pixel_search_for_neighbors (bool use_pbc)
4 {
5     pixel_grid * all_pixels;    // pointer to the pixel grid for to analyze
6     int pix, pjx;              // integer pixel ID numbers
7     int aid, bid;              // integer loop atom numbers
8     int pid;
9     int l_start, l_end;        // integer loop modifier
10    pixel * pix_i, * pix_j;     // pointers on pixel data structure
11    pixel_atom * at_i, * at_j;  // pointers on pixel_atom data structure
12    distance Dij;               // distance data structure
13
14    all_pixels = prepare_pixel_grid (use_pbc);
15    // note that the pixel grid 'all_pixels' must be prepared before the following
16    // for all pixels in the grid
17    for ( pix = 0 ; pix < all_pixels->pixels ; pix ++ )
18    {
19        // setting 'pix_i' as pointer to pixel number 'pix'
20        pix_i = & all_pixels->pixel_list[pix];
21        // if pixel 'pix_i' contains atom(s)
22        if ( pix_i->patoms )
23        {
24            // search for neighbor pixels of 'pix_i'
25            find_pixel_neighbors ( use_pbc, all_pixels, pix_i );
26            // testing all 'pix_i' neighbor pixels
27            for ( pid = 0 ; pid < pix_i->neighbors ; pid ++ )
28            {
29                pjx = pix_i->pixel_neighbors[pid];
30                // setting 'pix_j' as pointer to pixel number 'pjx'
31                pix_j = & all_pixels->pixel_list[pjx];
32                // checking pixel 'pix_j' if it:
33                // - contains atom(s)
34                // - was not tested, otherwise the analysis would have been performed already
35                if ( pix_j->patoms && ! pix_j->tested )
36                {
37                    // if 'pix_i' and 'pix_j' are the same, only test pair of different atoms
38                    l_end = (pjx != pix) ? 0 : 1;
39                    // for all atom(s) in 'pix'
40                    for ( aid = 0 ; aid < pix_i->patoms - l_end ; aid ++ )
41                    {
42                        // set pointer to the first atom to test
43                        at_i = & pix_i->pix_atoms[aid];
44                        l_start = (pjx != pix) ? 0 : aid + 1;
45                        // for all atom(s) in 'pix_j'
46                        for ( bid = l_start ; bid < pix_j->patoms ; bid ++ )
47                        {
48                            // set pointer to the second atom to test
49                            at_j = & pix_j->pix_atoms[bid];
50                            // evaluate interatomic distance
51                            Dij = evaluate_distance (use_pbc, at_i, at_j);
52                            if ( Dij.length < cutoff_squared )
53                            {
54                                // this is a first neighbor bond !
55                            }
56                        }
57                    }
58                }
59            }
60            // store that pixel 'pix' was tested
61            pix_i->tested = TRUE;
62        }
63    }
64 }

```

Appendix B Commented FORTRAN90 code

B.1 FORTRAN90 code: data structures and global variables

```

1  ! global data structures and variables used in the next code sections
2  MODULE parameters
3
4  ! atom in pixel data structure
5  TYPE pixel_atom
6      INTEGER                                :: atom_id          ! the atom ID
7      REAL, DIMENSION(3)                    :: coord             ! the atom coordinates on x, y and z
8  END TYPE pixel_atom
9
10 ! pixel data structure
11 TYPE pixel
12     INTEGER                                :: pid                ! the pixel number
13     INTEGER, DIMENSION(3)                  :: p_xyz              ! the pixel coordinates in the grid
14     LOGICAL                                :: tested              ! was the pixel checked already
15     INTEGER                                :: patoms              ! number of atom(s) in pixel
16     TYPE(pixel_atom), DIMENSION(:), ALLOCATABLE :: pix_atoms      ! list of atom(s) in pixel, to be allocated
17     INTEGER                                :: neighbors            ! number of neighbors for pixel
18     INTEGER, DIMENSION(27)                 :: pixel_neighbors    ! the list of neighbor pixels, maximum 27
19 END TYPE pixel
20
21 ! pixel grid data structure
22 TYPE pixel_grid
23     INTEGER                                :: pixels              ! total number of pixels in the grid
24     INTEGER, DIMENSION(3)                  :: n_pix              ! number of pixel(s) on each axis
25     INTEGER                                :: n_xy                ! number of pixels in the plan xy
26     TYPE(pixel), DIMENSION(:), ALLOCATABLE, TARGET :: pixel_list  ! pointer to the pixels, to be allocated
27 END TYPE pixel_grid
28
29 ! distance data structure
30 TYPE distance
31     REAL                                    :: length              ! the distance in \AA\ squared
32     REAL, DIMENSION(3)                     :: Rij                 ! vector components of x, y and z
33 END TYPE distance
34
35 !
36 ! the following are considered to be provided by the user
37 !
38
39 ! model description
40 INTEGER                                :: atoms                  ! the total number of atom(s)
41 REAL, DIMENSION(atoms,3)                :: c_coord              ! list of Cartesian coordinates
42 REAL                                    :: cutoff                 ! the cutoff to define first neighbors
43 REAL                                    :: cutoff_squared          ! squared value of the cutoff to define first neighbors
44
45 ! model box description, if PBC are applied
46 REAL                                    :: volume                 ! the lattice volume
47 REAL, DIMENSION(3)                       :: l_params             ! lattice a, b and c
48 REAL, DIMENSION(3,3)                     :: cart_to_frac        ! Cartesian to fractional coordinates matrix
49 REAL, DIMENSION(3,3)                     :: frac_to_cart         ! fractional to Cartesian coordinates matrix
50
51 END MODULE parameters

```

B.2 FORTRAN90 code: set pixel periodic boundary condition shift

```

1 SUBROUTINE set_pbc_shift (grid, pixel_coord, pbc_shift)
2
3   USE parameters
4   IMPLICIT NONE
5
6   TYPE (pixel_grid), INTENT(IN)           :: grid           ! the pixel grid
7   INTEGER, DIMENSION(3), INTENT(IN)       :: pixel_coord    ! the pixel coordinates in the grid
8   INTEGER, DIMENSION(3,3,3), INTENT(INOUT) :: pbc_shift      ! the shift, correction, to be calculated
9
10  pbc_shift(:, :, :) = 0                      ! initialization without any shift
11
12  if ( pixel_coord(1) .eq. 0 ) then           ! pixel position on 'x' is min
13    pbc_shift(1, :, :) = grid%n_pix(1)
14  else if ( pixel_coord(1) .eq. grid%n_pix(1)-1 ) then ! pixel position on 'x' is max
15    pbc_shift(3, :, :) = - grid%n_pix(1)
16  endif
17
18  if ( pixel_coord(2) .eq. 0 ) then           ! pixel position on 'y' is min
19    pbc_shift(:, 1, :) = pbc_shift(:, 1, :) + grid%n_xy
20  else if ( pixel_coord(2) .eq. grid%n_pix(2)-1 ) then ! pixel position on 'y' is max
21    pbc_shift(:, 3, :) = pbc_shift(:, 3, :) - grid%n_xy
22  endif
23
24  if ( pixel_coord(3) .eq. 0 ) then           ! pixel position on 'z' is min
25    pbc_shift(:, :, 1) = pbc_shift(:, :, 1) + grid%pixels
26  else if ( pixel_coord(3) .eq. grid%n_pix(3)-1 ) then ! pixel position on 'z' is max
27    pbc_shift(:, :, 3) = pbc_shift(:, :, 3) - grid%pixels
28  endif
29
30 END SUBROUTINE set_pbc_shift

```

B.3 FORTRAN90 code: add atom to pixel

```

1 SUBROUTINE add_atom_to_pixel (the_pixel, pixel_coord, atom_id, atom_coord)
2
3   USE parameters
4   IMPLICIT NONE
5
6   TYPE (pixel), INTENT(INOUT)              :: the_pixel    ! pointer to the pixel
7   INTEGER, DIMENSION(3), INTENT(IN)        :: pixel_coord  ! the pixel coordinates in the grid
8   INTEGER, INTENT(IN)                     :: atom_id       ! the atom ID number
9   INTEGER                                  :: axis           ! loop iterator axis id (1=x, 2=y, 3=z)
10  REAL, DIMENSION(3), INTENT(IN)           :: atom_coord   ! the atomic coordinates
11  TYPE(pixel_atom), DIMENSION(:), ALLOCATABLE :: tmp_atoms
12
13  if (the_pixel%patoms .eq. 0) then
14    ! if the pixel do not contains any atom yet, then save its coordinates in the grid
15    do axis = 1, 3
16      the_pixel%p_xyz(axis) = pixel_coord(axis)
17    enddo
18    ! allocate the memory to store the first pixel_atom information
19    allocate(the_pixel%pix_atoms(1))
20  else
21    ! otherwise reallocate memory to store the new pixel_atom information
22    allocate(tmp_atoms(the_pixel%patoms+1))
23    tmp_atoms(1:the_pixel%patoms) = the_pixel%pix_atoms
24    call move_alloc (tmp_atoms, the_pixel%pix_atoms)
25    deallocate (tmp_atoms)
26  endif
27  ! increment the number of atom(s) in the pixel
28  the_pixel%patoms = the_pixel%patoms + 1
29  the_pixel%pix_atoms(the_pixel%patoms)%atom_id = atom_id
30  do axis = 1, 3
31    the_pixel%pix_atoms(the_pixel%patoms)%coord(axis) = atom_coord(axis)
32  enddo
33
34 END SUBROUTINE add_atom_to_pixel

```

B.4 FORTRAN90 code: adjusting the number of pixels in the grid

```

1 SUBROUTINE adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size)
2
3   USE parameters
4   IMPLICIT NONE
5
6   LOGICAL, INTENT(IN)           :: use_pbc           ! flag to set if PBC are used or not
7   TYPE (pixel_grid), INTENT(INOUT) :: grid           ! the pixel grid to prepare
8   REAL, DIMENSION(3), INTENT(IN) :: cmin, cmax       ! real precision coordinates min, max
9   REAL, INTENT(INOUT)          :: pixel_size         ! the initial pixel size
10  REAL                          :: targetdp=1.85      ! target atom density per pixel
11  REAL                          :: rhonum             ! number density
12  REAL                          :: rhopix             ! number density by pixel
13  REAL                          :: mpsize             ! modifier of the pixel size
14  INTEGER                       :: axis                ! loop iterator axis id (1=x, 2=y, 3=z)
15
16  rhonum = atoms
17  if (use_pbc) then
18    ! if PBC are used then use the exact volume
19    rhonum = rhonum / volume
20  else
21    ! otherwise approximate a cubic box
22    do axis = 1, 3 ! for x, y and z
23      rhonum = rhonum / (cmax(axis) - cmin(axis))
24    enddo
25  endif
26
27  ! if the number density is < 0.01 atom / Å³
28  if (rhonum .lt. 0.01) then
29
30    rhopix = atoms
31    do axis = 1, 3 ! for x, y and z
32      rhopix = rhopix / grid%n_pix(axis)
33    enddo
34    mpsize = (targetdp/rhopix)**(1.0/3.0)
35    if (mpsize .gt. 1.0) then
36      ! we modify only the pixel size if the cutoff increases
37      ! otherwise we would increase the number of pixels
38      pixel_size = pixel_size*mpsize
39      do axis= 1, 3 ! for x, y and z
40        if ( use_pbc ) then
41          grid%n_pix(axis) = INT(l_params(axis) / pixel_size)
42        else
43          grid%n_pix(axis) = INT((cmax(axis) - cmin(axis)) / pixel_size)
44        endif
45      enddo
46    endif
47  endif
48
49  do axis = 1, 3 ! for x, y and z
50    ! correction if the number of pixel(s) on 'axis' is too small
51    if ( grid%n_pix(axis) .lt. 3 ) then
52      grid%n_pix(axis) = 1
53    endif
54  enddo
55
56  END SUBROUTINE adjust_pixel_numbers

```

B.5 FORTRAN90 code: preparation of the pixel grid

```

1 ! preparation of the pixel grid
2 SUBROUTINE prepare_pixel_grid (use_pbc, grid)
3
4 USE parameters
5 IMPLICIT NONE
6
7 LOGICAL, INTENT(IN)          :: use_pbc          ! flag to set if PBC are used or not
8 TYPE (pixel_grid), INTENT(OUT) :: grid          ! the pixel grid to prepare
9 INTEGER                     :: axis              ! loop iterator axis id (1=x, 2=y, 3=z)
10 INTEGER                     :: aid               ! loop iterator atom number (1, atoms)
11 INTEGER                     :: pixel_num         ! pixel number in the grid
12 INTEGER, DIMENSION(3)       :: pixel_pos        ! pixel coordinates in the grid
13 REAL, DIMENSION(3)          :: cmin, cmax        ! real precision coordinates min, max
14 REAL, DIMENSION(3)          :: f_coord          ! real precision fractional coordinates
15 REAL                        :: pixel_size        ! the size of the pixel
16
17 pixel_size = cutoff + 0.5          ! set a pixel size slightly higher than the cutoff
18 if ( use_pbc ) then                ! using periodic boundary conditions
19     do axis = 1 , 3                ! for x, y and z
20         ! number of pixel(s) on 'axis'
21         grid%n_pix(axis) = INT(l_params(axis) / pixel_size)
22     enddo
23 else                                ! without periodic boundary conditions
24     do axis = 1 , 3
25         cmin(axis) = c_coord(1,axis)
26         cmax(axis) = c_coord(1,axis)
27     enddo
28     do aid = 2 , atoms              ! for all atoms
29         do axis = 1 , 3              ! for x, y and z
30             cmin(axis) = min(cmin(axis), c_coord(aid,axis))
31             cmax(axis) = max(cmax(axis), c_coord(aid,axis))
32         enddo
33     enddo
34     do axis = 1 , 3                ! for x, y and z
35         ! number of pixel(s) on 'axis'
36         grid%n_pix(axis) = INT((cmax(axis) - cmin(axis))/pixel_size)
37     enddo
38 endif
39
40 call adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size)
41
42 grid%xy = grid%n_pix(1) * grid%n_pix(2)      ! number of pixels on the plan 'xy'
43 grid%pixels = grid%xy * grid%n_pix(3)        ! total number of pixels in the grid
44
45 allocate (grid%pixel_list(grid%pixels))
46 do pixel_num = 1 , grid%pixels
47     grid%pixel_list(pixel_num)%pid = pixel_num
48     grid%pixel_list(pixel_num)%patoms = 0
49     grid%pixel_list(pixel_num)%tested = .false.
50 enddo
51 if ( use_pbc ) then                ! using periodic boundary conditions
52     do aid = 1 , atoms              ! for all atoms
53         f_coord = MATMUL ( c_coord(aid,:), cart_to_frac )
54         do axis = 1 , 3              ! for x, y and z
55             f_coord(axis) = f_coord(axis) - floor(f_coord(axis))
56             pixel_pos(axis) = INT(f_coord(axis) * grid%n_pix(axis))
57         enddo
58         pixel_num = pixel_pos(1) + pixel_pos(2) * grid%n_pix(1) + pixel_pos(3) * grid%xy
59         call add_atom_to_pixel (grid%pixel_list(pixel_num+1), pixel_pos, aid, f_coord)
60     enddo
61 else                                ! without periodic boundary conditions
62     do aid = 1 , atoms              ! for all atoms
63         do axis = 1 , 3              ! for x, y and z
64             if (grid%n_pix(axis) .eq. 1) then
65                 pixel_pos(axis) = 0
66             else
67                 pixel_pos(axis) = INT((c_coord(aid,axis) - cmin(axis))/pixel_size)
68             endif
69         enddo
70         pixel_num = pixel_pos(1) + pixel_pos(2) * grid%n_pix(1) + pixel_pos(3) * grid%xy
71         call add_atom_to_pixel (grid%pixel_list(pixel_num+1), pixel_pos, aid, c_coord(aid,:))
72     enddo
73 endif
74
75 END SUBROUTINE prepare_pixel_grid

```

B.6 FORTRAN90 code: finding pixel neighbors

```

1 SUBROUTINE find_pixel_neighbors (use_pbc, the_grid, the_pix)
2
3   USE parameters
4   IMPLICIT NONE
5
6   LOGICAL, INTENT(IN)           :: use_pbc           ! flag to set if PBC are used or not
7   TYPE (pixel_grid), INTENT(INOUT) :: the_grid       ! pointer to the pixel grid
8   TYPE (pixel), POINTER, INTENT(INOUT) :: the_pix     ! pointer to the pixel
9   INTEGER                      :: axis              ! loop iterator axis id (1=x, 2=y, 3=z)
10  INTEGER                      :: x_pos, y_pos, z_pos ! neighbor position on x, y and z
11  INTEGER, DIMENSION(3)        :: l_start = (/1, 1, 1/) ! loop iterators starting value
12  INTEGER, DIMENSION(3)        :: l_end = (/3, 3, 3/) ! loop iterators ending value
13  INTEGER, DIMENSION(3)        :: pmod = (/ -1, 0, 1/) ! position modifiers
14  INTEGER                      :: nnp               ! number of neighbors for pixel
15  INTEGER                      :: nid               ! neighbor id for pixel
16  INTEGER, DIMENSION(3,3,3)    :: pbc_shift        ! shift, correction, due to PBC
17  LOGICAL                      :: boundary=.false.   ! is pixel on the boundary of the grid
18  LOGICAL                      :: keep_neighbor = .true. ! keep or not neighbor during analysis
19
20  if ( use_pbc ) then
21    call set_pbc_shift (the_grid, the_pix%p_xyz, pbc_shift)
22  else
23    do axis = 1, 3
24      if ( the_pix%p_xyz(axis) .eq. 0 .or. the_pix%p_xyz(axis) .eq. the_grid%n_pix(axis) - 1 ) then
25        boundary = .true.
26      endif
27    enddo
28  endif
29  do axis = 1, 3
30    if ( the_grid%n_pix(axis) .eq. 1 ) then
31      ! in the grid there is a single pixel in the 'axis' direction
32      l_start(axis) = 2
33      l_end(axis) = 2
34    endif
35  enddo
36  nnp = 0
37  do x_pos = l_start(1), l_end(1)
38    do y_pos = l_start(2), l_end(2)
39      do z_pos = l_start(3), l_end(3)
40        keep_neighbor = .true.
41        if ( .not. use_pbc .and. boundary ) then
42          if ( the_pix%p_xyz(1) .eq. 0 .and. x_pos .eq. 1 ) then
43            keep_neighbor = .false.
44          else if ( the_pix%p_xyz(1) .eq. the_grid%n_pix(1)-1 .and. x_pos .eq. 3 ) then
45            keep_neighbor = .false.
46          else if ( the_pix%p_xyz(2) .eq. 0 .and. y_pos .eq. 1 ) then
47            keep_neighbor = .false.
48          else if ( the_pix%p_xyz(2) .eq. the_grid%n_pix(2)-1 .and. y_pos .eq. 3 ) then
49            keep_neighbor = .false.
50          else if ( the_pix%p_xyz(3) .eq. 0 .and. z_pos .eq. 1 ) then
51            keep_neighbor = .false.
52          else if ( the_pix%p_xyz(3) .eq. the_grid%n_pix(3)-1 .and. z_pos .eq. 3 ) then
53            keep_neighbor = .false.
54          endif
55        endif
56        if ( keep_neighbor ) then
57          ! evaluating neighbor pixel number in the grid
58          nid = the_pix%pid + pmod(x_pos) + pmod(y_pos) * the_grid%n_pix(1) + pmod(z_pos) * the_grid%n_xy
59          if ( use_pbc ) then
60            ! corrections if required in the case where PBC are applied
61            nid = nid + pbc_shift(x_pos, y_pos, z_pos)
62          endif
63          the_pix%pixel_neighbors(nnp) = nid
64          nnp = nnp + 1
65        endif
66      enddo
67    enddo
68  enddo
69  ! total number of neighbors for pixel 'the_pix'
70  the_pix%neighbors = nnp
71
72 END SUBROUTINE find_pixel_neighbors

```

B.7 FORTRAN90 code: inter-atomic distance calculation

```

1  ! evaluating the interatomic distance between 2 pixel atoms
2  SUBROUTINE evaluate_distance (use_pbc, at_i, at_j, dist)
3
4  USE parameters
5  IMPLICIT NONE
6
7  LOGICAL, INTENT(IN)                :: use_pbc    ! flag to set if PBC are used or not
8  TYPE (pixel_atom), POINTER, INTENT(IN) :: at_i    ! first pixel atom
9  TYPE (pixel_atom), POINTER, INTENT(IN) :: at_j    ! second pixel atom
10 TYPE (distance), INTENT(INOUT)      :: dist      ! calculation results
11 INTEGER                            :: axis        ! loop iterator axis id (1=x, 2=y, 3=z)
12
13 do axis = 1, 3
14     dist%Rij(axis) = at_i%coord(axis) - at_j%coord(axis)
15 enddo
16 if ( use_pbc ) then
17     ! then the pixel_atom's coordinates are in corrected fractional format
18     do axis = 1, 3
19         dist%Rij(axis) = dist%Rij(axis) - AnINT(dist%Rij(axis))
20     enddo
21     ! transform back to Cartesian coordinates
22     dist%Rij = MATMUL( dist%Rij, frac_to_cart )
23 endif
24
25 dist%length = 0.0
26 do axis = 1, 3
27     dist%length = dist%length + dist%Rij(axis) * dist%Rij(axis)
28 enddo
29 ! returning the 'distance' data structure that contains:
30 ! - the squared value for Dij: no time consuming square root calculation !
31 ! - the components of the distance vector on x, y and z
32 END SUBROUTINE evaluate_distance

```

B.8 FORTRAN90 code: pixel search for first neighbor atoms

```

1 SUBROUTINE pixel_search_for_neighbors (use_pbc)
2
3   USE parameters
4   IMPLICIT NONE
5
6   LOGICAL, INTENT(IN)  :: use_pbc                ! flag to set if PBC are used or not
7   TYPE (pixel_grid)   :: all_pixels              ! the pixel grid to analyze
8   INTEGER             :: pix, pjx                ! integer pixel ID numbers
9   INTEGER             :: aid, bid                ! integer loop atom numbers
10  INTEGER             :: l_start, l_end          ! integer loop modifier
11  INTEGER             :: pid
12  TYPE (pixel_atom), POINTER :: pix_i, pix_j      ! pointers on pixel data structure
13  TYPE (atom), POINTER  :: at_i, at_j            ! pointers on pixel_atom data structure
14  TYPE (distance)       :: Dij                  ! distance data structure
15
16  call prepare_pixel_grid (use_pbc, all_pixels)
17  ! note that the pixel grid 'all_pixels' must be prepared before the following
18  ! for all pixels in the grid
19  do pix = 1, all_pixels%pixels
20    ! setting 'pix_i' as pointer to pixel number 'pix'
21    pix_i => all_pixels%pixel_list(pix)
22    ! if pixel 'pix_i' contains atom(s)
23    if ( pix_i%patoms .gt. 0 ) then
24
25      ! search for pixel neighbors of 'pix_i'
26      call find_pixel_neighbors (use_pbc, all_pixels, pix_i)
27
28      ! testing all 'pix_i' neighbor pixels
29      do pid = 1, pix_i%neighbors
30        pjx = pix_i%pixel_neighbors(pid)
31        ! setting 'pix_j' as pointer to pixel number 'pjx'
32        pix_j => all_pixels%pixel_list(pjx)
33        ! checking pixel 'pix_j' if it:
34        ! - contains atom(s)
35        ! - was not tested, otherwise the analysis would have been performed already
36        if ( pix_j%patoms .gt. 0 .and. .not. pix_j%tested ) then
37          ! If 'pix_i' and 'pix_j' are the same, only test pair of different atoms
38          if ( pjx .eq. pix ) then
39            l_end = 1
40          else
41            l_end = 0
42          endif
43          ! for all atom(s) in 'pix_i'
44          do aid = 1, pix_i%patoms - l_end
45            ! set pointers to the first atom to test
46            at_i => pix_i%pix_atoms(aid)
47            if ( pjx .eq. pix ) then
48              l_start = aid + 1
49            else
50              l_start = 1
51            endif
52            ! for all atom(s) in 'pix_j'
53            do bid = l_start, pix_j%patoms
54              ! set pointers to the second atom to test
55              at_j => pix_j%pix_atoms(bid)
56              ! evaluate interatomic distance
57              call evaluate_distance (use_pbc, at_i, at_j, Dij)
58              if ( Dij%length .lt. cutoff_squared ) then
59                ! this is a first neighbor bond !
60                endif
61              enddo
62            enddo
63          endif
64        enddo
65        ! store that pixel 'pix_i' was tested
66        pix_i%tested = .true.
67      endif
68    enddo
69  enddo
70 END SUBROUTINE pixel_search_for_neighbors

```


Appendix C Commented Python code

C.1 Python code: data structures and global variables

```

1 # Global data structures and variables used in the next code sections
2 import numpy as np
3
4 # atom in pixel data structure
5 class PixelAtom:
6     def __init__(self, atom_id=0, coord=None):
7         self.atom_id = atom_id # the atom ID
8         self.coord = np.zeros(3) if coord is None else np.array(coord) # the atom coordinates on x, y and z
9
10 # pixel data structure
11 class Pixel:
12     def __init__(self, pid=0, p_xyz=None, tested=False, patoms=0, pix_atoms=None, neighbors=0):
13         self.pid = pid # the pixel number
14         self.p_xyz = np.zeros(3) if p_xyz is None else np.array(p_xyz) # the pixel coordinates in the grid
15         self.tested = tested # was the pixel checked already
16         self.patoms = patoms # number of atom(s) in pixel
17         self.pix_atoms = [] if pix_atoms is None else pix_atoms # list of atom(s) in the pixel
18         self.neighbors = neighbors # number of neighbors for pixel
19         self.pixel_neighbors = np.zeros(27, dtype=int) # the list of neighbor pixels, maximum 27
20
21 # pixel grid data structure
22 class PixelGrid:
23     def __init__(self, pixels=0, n_pix=None, n_xy=0, pixel_list=None):
24         self.pixels = pixels # total number of pixels in the grid
25         self.n_pix = np.zeros(3, dtype=int) if n_pix is None else np.array(n_pix) # pixel(s) on each axis
26         self.n_xy = n_xy # number of pixels in the plan xy
27         self.pixel_list = [] if pixel_list is None else pixel_list # pointer to the pixels, to be allocated
28
29 # bond distance data structure
30 class Distance:
31     def __init__(self, length=0.0, Rij=None):
32         self.length = length # the distance in Å squared
33         self.Rij = np.zeros(3) if Rij is None else np.array(Rij) # vector components of x, y and z
34
35 #
36 # the following are considered to be provided by the user
37 #
38
39 # model description
40 atoms = 0 # the total number of atom(s)
41 c_coord = None # list of Cartesian coordinates: c_coord[atoms][3]
42 cutoff = 0.0 # the cutoff to define atomic bond(s)
43 cutoff_squared = 0.0 # squared value for the cutoff
44
45 # model box description, if PBC are applied
46 volume = 0.0 # the lattice volume
47 l_params = np.zeros(3) # lattice a, b and c
48 cart_to_frac = np.zeros((3, 3)) # Cartesian to fractional coordinates matrix
49 frac_to_cart = np.zeros((3, 3)) # fractional to Cartesian coordinates matrix

```

C.2 Python code: set pixel periodic boundary condition shift

```

1 # adjust, if needed, the shift to search for pixel neighbor(s) using PBC
2 # - PixelGrid pixel_grid      : the pixel grid
3 # - int pixel_coord[3]        : the pixel coordinates in the grid
4 # - int pbc_shift[3][3][3]    : the shift, correction, to be calculated
5 def set_pbc_shift(pixel_grid : PixelGrid, pixel_coord : np.ndarray, pbc_shift : np.ndarray):
6     # Initialize pbc_shift to zero
7     for x_pos in range(3):
8         for y_pos in range(3):
9             for z_pos in range(3):
10                 pbc_shift[x_pos][y_pos][z_pos] = 0           # initialization without any shift
11
12     if pixel_coord[0] == 0:                                   # pixel position on 'x' is min
13         for y_pos in range(3):
14             for z_pos in range(3):
15                 pbc_shift[0][y_pos][z_pos] = pixel_grid.n_pix[0]
16
17     elif pixel_coord[0] == pixel_grid.n_pix[0] - 1: # pixel position on 'x' is max
18         for y_pos in range(3):
19             for z_pos in range(3):
20                 pbc_shift[2][y_pos][z_pos] = -pixel_grid.n_pix[0]
21
22     if pixel_coord[1] == 0:                                   # pixel position on 'y' is min
23         for x_pos in range(3):
24             for z_pos in range(3):
25                 pbc_shift[x_pos][0][z_pos] += pixel_grid.n_xy
26
27     elif pixel_coord[1] == pixel_grid.n_pix[1] - 1: # pixel position on 'y' is max
28         for x_pos in range(3):
29             for z_pos in range(3):
30                 pbc_shift[x_pos][2][z_pos] -= pixel_grid.n_xy
31
32     if pixel_coord[2] == 0:                                   # pixel position on 'z' is min
33         for x_pos in range(3):
34             for y_pos in range(3):
35                 pbc_shift[x_pos][y_pos][0] += pixel_grid.pixels
36
37     elif pixel_coord[2] == pixel_grid.n_pix[2] - 1: # pixel position on 'z' is max
38         for x_pos in range(3):
39             for y_pos in range(3):
40                 pbc_shift[x_pos][y_pos][2] -= pixel_grid.pixels

```

C.3 Python code: add atom to pixel

```

1 def add_atom_to_pixel(the_pixel: Pixel, pixel_coord: np.ndarray, atom_id: int, atom_coord: np.ndarray):
2     if not the_pixel.patoms:
3         # if the pixel do not contains any atom yet, then save its coordinates in the grid
4         for axis in range(3):
5             the_pixel.p_xyz[axis] = pixel_coord[axis]
6
7     new_atom = PixelAtom(atom_id, atom_coord)
8     the_pixel.pix_atoms.append(new_atom)
9
10    # increment the number of atom(s) in the pixel
11    the_pixel.patoms += 1

```

C.4 Python code: adjusting the number of pixels in the grid

```

1 def adjust_pixel_numbers (use_pbc : bool, grid : PixelGrid, cmin : np.ndarray, cmax : np.ndarray, pixel_size : float):
2
3     targetdp = 1.85 # target atom density per pixel
4     rhonum = atoms
5     if use_pbc:
6         rhonum = rhonum / volume
7     else:
8         for axis in range(3): # for x, y and z
9             rhonum = rhonum / (cmax[axis] - cmin[axis])
10
11     if rhonum < 0.01:
12         rhopix = atoms
13         for axis in range(3): # for x, y and z
14             rhopix = rhopix / grid.n_pix[axis]
15
16     mpsize = (targetdp / rhopix ) **(1.0/3.0)
17     if mpsize > 1.0:
18         # we modify only the pixel size if the cutoff increases
19         # otherwise we would increase the number of pixels
20         pixel_size = pixel_size * mpsize
21         for axis in range(3): # for x, y and z
22             if use_pbc:
23                 grid.n_pix[axis] = int(l_params[axis] / pixel_size)
24             else
25                 grid.n_pix[axis] = int((cmax[axis] - cmin[axis]) / pixel_size)
26
27     for axis in range(3): # for x, y and z
28         # correction if the number of pixels on 'axis' is too small
29         grid.n_pix[axis] = 1 if grid.n_pix[axis] < 3 else grid.n_pix[axis]
30
31     return pixel_size

```

C.5 Python code: preparation of the pixel grid

```

1 # preparation of the pixel grid
2 # - bool use_pbc : flag to set if PBC are used or not
3 def prepare_pixel_grid(use_pbc : bool):
4     grid = PixelGrid()                                # create a new pixel grid
5     cmin = [float('inf')] * 3                          # initialize to infinity
6     cmax = [-float('inf')] * 3                        # initialize to negative infinity
7     pixel_pos = np.zeros(3, dtype=int)
8     pixel_size = cutoff + 0.5;                        # set a pixel size slightly higher than the cutoff
9
10    if use_pbc:                                         # using periodic boundary conditions
11        for axis in range(3):                          # for x, y and z
12            # number of pixels on 'axis'
13            grid.n_pix[axis] = int(l_params[axis] / pixel_size)
14    else:                                               # without periodic boundary conditions
15        for axis in range(3):
16            cmin[axis] = cmax[axis] = c_coord[0][axis]
17        for aid in range(1, atoms):                   # for all atoms
18            for axis in range(3):                       # for x, y and z
19                cmin[axis] = min(cmin[axis], c_coord[aid][axis])
20                cmax[axis] = max(cmax[axis], c_coord[aid][axis])
21        for axis in range(3):                           # for x, y and z
22            # number of pixels on 'axis'
23            grid.n_pix[axis] = int((cmax[axis] - cmin[axis]) / pixel_size)
24
25    pixel_size = adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size)
26
27    grid.n_xy = grid.n_pix[0] * grid.n_pix[1]          # number of pixels on the plan 'xy'
28    grid.pixels = grid.n_xy * grid.n_pix[2]           # total number of pixels in the grid
29
30    grid.pixel_list = [Pixel(pid=i) for i in range(grid.pixels)]
31
32    if use_pbc:                                         # using periodic boundary conditions
33        for aid in range(atoms):                       # for all atoms
34            # with 'matrix_multiplication' a user defined function to perform the operation
35            f_coord = matrix_multiplication(cart_to_frac, c_coord[aid])
36            for axis in range(3):                       # for x, y and z
37                f_coord[axis] = f_coord[axis] - np.floor(f_coord[axis])
38                pixel_pos[axis] = int(f_coord[axis] * grid.n_pix[axis])
39            pixel_num = pixel_pos[0] + pixel_pos[1] * grid.n_pix[0] + pixel_pos[2] * grid.n_xy
40            add_atom_to_pixel(grid.pixel_list[pixel_num], pixel_pos, aid, f_coord)
41    else:                                               # without periodic boundary conditions
42        for aid in range(atoms):                       # for all atoms
43            for axis in range(3):                       # for x, y and z
44                pixel_pos[axis] = int((c_coord[aid][axis] - cmin[axis]) / pixel_size)
45            pixel_num = pixel_pos[0] + pixel_pos[1] * grid.n_pix[0] + pixel_pos[2] * grid.n_xy
46            add_atom_to_pixel(grid.pixel_list[pixel_num], pixel_pos, aid, c_coord[aid])
47
48    return grid

```

C.6 Python code: finding pixel neighbors

```

1 # Finding neighbor pixels for pixel in the grid
2 # - bool use_pbc : flag to set if PBC are used or not
3 # - PixelGrid * the_grid : pointer to the pixel grid
4 # - Pixel * the_pix : pointer to the pixel with neighbors to be found
5 def find_pixel_neighbors(use_pbc : bool, the_grid : PixelGrid, the_pix : Pixel):
6     boundary = False # is pixel on the boundary of the grid
7     keep_neighbor = True # keep or not neighbor during analysis
8     l_start = [0, 0, 0] # loop iterators starting value
9     l_end = [3, 3, 3] # loop iterators ending value
10    pmod = [-1, 0, 1] # position modifiers
11    pbc_shift = np.zeros((3, 3, 3), dtype=int) # shift for pixel neighbor number due to PBC
12
13    # check if PBC are used
14    if use_pbc:
15        # set PBC correction based on grid and pixel data
16        set_pbc_shift(the_grid, the_pix.p_xyz, pbc_shift)
17    else:
18        for axis in range(3):
19            if the_pix.p_xyz[axis] == 0 or the_pix.p_xyz[axis] == the_grid.n_pix[axis] - 1:
20                # if min or max on 'axis' then 'the_pix' is on the edge of the box
21                boundary = True
22
23    # adjust the loop start and end based on the grid dimensions
24    for axis in range(3):
25        if the_grid.n_pix[axis] == 1:
26            # in the grid there is a single pixel in the 'axis' direction
27            l_start[axis] = 1
28            l_end[axis] = 1
29
30    nnp = 0 # number of neighbors
31    for x_pos in range(l_start[0], l_end[0]):
32        for y_pos in range(l_start[1], l_end[1]):
33            for z_pos in range(l_start[2], l_end[2]):
34                keep_neighbor = True
35
36                if not use_pbc and boundary:
37                    if the_pix.p_xyz[0] == 0 and x_pos == 0:
38                        keep_neighbor = False
39                    elif the_pix.p_xyz[0] == the_grid.n_pix[0]-1 and x_pos == 2:
40                        keep_neighbor = False
41                    elif the_pix.p_xyz[1] == 0 and y_pos == 0:
42                        keep_neighbor = False
43                    elif the_pix.p_xyz[1] == the_grid.n_pix[1]-1 and y_pos == 2:
44                        keep_neighbor = False
45                    elif the_pix.p_xyz[2] == 0 and z_pos == 0:
46                        keep_neighbor = False
47                    elif the_pix.p_xyz[2] == the_grid.n_pix[2]-1 and z_pos == 2:
48                        keep_neighbor = False
49
50                if keep_neighbor:
51                    # evaluating neighbor pixel number in the grid
52                    nid = the_pix.pid + pmod[x_pos] + pmod[y_pos] * the_grid.n_pix[0] + pmod[z_pos] * the_grid.n_xy
53                    if use_pbc:
54                        # corrections if required in the case where PBC are applied
55                        nid += pbc_shift[x_pos][y_pos][z_pos]
56                    the_pix.pixel_neighbors[nnp] = nid
57                    nnp += 1
58
59    # total number of neighbors for pixel 'the_pix'
60    the_pix.neighbors = nnp

```

C.7 Python code: inter-atomic distance calculation

```

1 # evaluating the interatomic distance between 2 pixel atoms
2 def evaluate_distance(use_pbc : bool, at_i : PixelAtom, at_j : PixelAtom):
3     dist = Distance() # placeholder for the distance data structure
4     Rij = np.zeros(3) # initialize the distance vector
5     # calculating the distance components between atoms
6     for axis in range(3):
7         Rij[axis] = at_i.coord[axis] - at_j.coord[axis]
8
9     if use_pbc:
10        # then the PixelAtom's coordinates are in corrected fractional format
11        for axis in range(3):
12            Rij[axis] = Rij[axis] - round(Rij[axis])
13
14        # transform back to Cartesian coordinates
15        # matrix_multiplication is assumed to be defined elsewhere
16        Rij = matrix_multiplication(frac_to_cart, Rij)
17
18    # calculating the squared distance (no square root for efficiency)
19    dist.length = np.sum(Rij ** 2)
20    dist.Rij = Rij # store the distance vector components
21
22    # returning the 'distance' data structure that contains:
23    # - the squared value for Dij
24    # - the components of the distance vector on x, y, and z
25    return dist

```

C.8 Python code: pixel search for first neighbor atoms

```

1 def pixel_search_for_neighbors(use_pbc : bool):
2     # pointer to the pixel grid for analysis
3     all_pixels = prepare_pixel_grid(use_pbc)
4
5     # for all pixels in the grid
6     for pix in range(all_pixels.pixels):
7         # setting pix_i as pointer to pixel number pix
8         pix_i = all_pixels.pixel_list[pix]
9
10        # if pixel pix_i contains atom(s)
11        if pix_i.patoms:
12            # search for neighbor pixels
13            find_pixel_neighbors(use_pbc, all_pixels, pix_i)
14
15        # testing all pix_i neighbor pixels
16        for pid in range(pix_i.neighbors):
17            p_jx = pix_i.pixel_neighbors[pid]
18            # Setting pix_j as pointer to pixel number p_jx
19            pix_j = all_pixels.pixel_list[p_jx]
20
21            # checking pixel pix_j if it:
22            # - contains atom(s)
23            # - was not tested, otherwise the analysis would have been performed already
24            if pix_j.patoms and not pix_j.tested:
25                # If pix_i and pix_j are the same, only test pair of different atoms
26                end = 0 if p_jx != pix else 1
27
28                # for all atom(s) in pix_i
29                for aid in range(pix_i.patoms - end):
30                    # Set pointer to the first atom to test
31                    at_i = pix_i.pix_atoms[aid]
32                    start = 0 if p_jx != pix else aid + 1
33
34                    # for all atom(s) in pix_j
35                    for bid in range(start, pix_j.patoms):
36                        # Set pointer to the second atom to test
37                        at_j = pix_j.pix_atoms[bid]
38
39                        # evaluate interatomic distance
40                        Dij = evaluate_distance(use_pbc, at_i, at_j)
41
42                        if Dij.length < cutoff_squared:
43                            # This is a bond!
44                            pass
45
46                # store that pixel pix_i was tested
47                pix_i.tested = True

```